'''

1. Linear Regression – Full Notes (Beginner → Intermediate)

Linear Regression is a Supervised Machine Learning Algorithm used to predict continuous numerical values.

Examples:

- House Price Prediction
- Salary Prediction
- Temperature Forecast
- Sales Prediction

What is Linear Regression?

Linear Regression finds the linear relationship between input and output.

Example:

Area (x)	Price (y)
1000	200
1500	300
2000	400

As area increases, price increases.
When plotted on a graph, it forms a straight line.

Formula:

$y = mx + c$

Where:

- y → Predicted value
- x → Input feature
- m → Slope
- c → Intercept

2. Simple Linear Regression

Used when there is only one feature.

Formula:

$y = mx + c$

Example:

$y = 2x + 5$

If:

$x = 10$

Then:

$y = 2(10) + 5$

$y = 25$

3. Slope Formula

$$m = \Sigma(x_i - \bar{x})(y_i - \bar{y}) / \Sigma(x_i - \bar{x})^2$$

Where:

- x_i = Input value
- y_i = Output value
- $\bar{x}$ = Mean of x
- $\bar{y}$ = Mean of y

Given:

$n = 3$

$x = [1000, 1500, 2000]$

$y = [200, 300, 400]$

Step 1: Calculate the means

$\bar{x} = (1000 + 1500 + 2000) / 3$

$= 4500 / 3$

$= 1500$

$\bar{y} = (200 + 300 + 400) / 3$

$= 900 / 3$

$= 300$

Step 2: Calculate the required values

x_i	y_i	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})(y_i - \bar{y})$	$(x_i - \bar{x})^2$
1000	200	-500	-100	50,000	250,000
1500	300	0	0	0	0
2000	400	500	100	50,000	250,000

$\Sigma(x_i - \bar{x})(y_i - \bar{y}) = 100,000$

$\Sigma(x_i - \bar{x})^2 = 500,000$

```
    Step 3: Calculate the slope

    m = 100,000 / 500,000
    m = 0.2

    Final Answer:
    Slope (m) = 0.2

4.  Intercept Formula
    c = ȳ - mx̄

    c = 300 - (0.2 * 1500)
    c = 300 - 300
    c = 0

    Final Answer:
    c = 0

5.  Prediction Formula
    ŷ = mx + c

    Where:
    ŷ = Predicted value of y
    m = Slope of the regression line
    x = Input (independent variable)
    c = Intercept (constant)

    Given:
    m = 0.2
    c = 0

    Therefore,

    ŷ = 0.2x + 0
    ŷ = 0.2x

    Example:
    If x = 2500,

    ŷ = 0.2 * 2500
    ŷ = 500
```

Predicted value = 500

6. Cost Function

Cost Function is used to measure how much error exists between the actual values and the predicted values.

The goal of Linear Regression is to minimize the cost (error).

Mean Squared Error (MSE):

$J(m,c) = (1/n)\ \Sigma(y_i - \hat{y}_i)^2$

Where:

- $J(m,c)$ = Cost Function (Error)
- n = Number of data points
- y_i = Actual value
- $\hat{y}_i$ = Predicted value
- m = Slope
- c = Intercept

Given:

$m = 0.2$

$c = 0$

Regression Equation:

$\hat{y} = 0.2x$

Data:

x	Actual (y)	Predicted (ŷ)
1000	200	200
1500	300	300
2000	400	400

Step 1: Calculate Errors

Actual (y)	Predicted (ŷ)	Error (y - ŷ)	Squared Error $(y - \hat{y})^2$
200	200	0	0
300	300	0	0
400	400	0	0

Step 2: Calculate the Cost Function

$$J(0.2,0) = (1/3)\,[(200-200)^2 + (300-300)^2 + (400-400)^2]$$

$$= (1/3)\,(0 + 0 + 0)$$

$$= 0/3$$

$$= 0$$

Final Answer:

Cost Function (MSE) = 0

Interpretation:
- MSE = 0 → Perfect prediction (No error)
- Lower MSE → Better model
- Higher MSE → More prediction error

7. MAE (Mean Absolute Error)

Mean Absolute Error (MAE) measures the average absolute difference between the actual values and the predicted values.

Formula:

$$MAE = (1/n)\,\Sigma|y_i - \hat{y}_i|$$

Where:
- MAE = Mean Absolute Error

- n = Number of data points
- y_i = Actual value
- $\hat{y}_i$ = Predicted value

Example:

Actual Values (y):
[10, 20]

Predicted Values (ŷ):
[12, 18]

Step 1: Calculate Absolute Errors

$|10 - 12| = 2$
$|20 - 18| = 2$

Step 2: Calculate MAE

$$MAE = (2 + 2) / 2$$

$$= 4 / 2$$

$$= 2$$

Final Answer:

MAE = 2

Interpretation:
- MAE = 0 → Perfect prediction
- Lower MAE → Better model
- Higher MAE → More prediction error

8. MSE (Mean Squared Error)

Mean Squared Error (MSE) measures the average of the squared differences between the actual values and the predicted values.

Formula:

```
MSE = (1/n) Σ(yi - ŷi)²

Where:
• MSE = Mean Squared Error
• n = Number of data points
• yi = Actual value
• ŷi = Predicted value

Example:

Actual Values (y):
[10, 20]

Predicted Values (ŷ):
[12, 18]

Step 1: Calculate Errors

10 - 12 = -2
20 - 18 = 2

Errors:
[-2, 2]

Step 2: Square the Errors

(-2)² = 4
(2)² = 4

Squared Errors:
[4, 4]

Step 3: Calculate MSE

MSE = (4 + 4) / 2

    = 8 / 2

    = 4

Final Answer:
```

MSE = 4

Interpretation:

• MSE = 0 → Perfect prediction

• Lower MSE → Better model

• Higher MSE → More prediction error

• MSE penalizes larger errors more because the errors are squared.

9. RMSE (Root Mean Squared Error)

Root Mean Squared Error (RMSE) is the square root of the Mean Squared Error (MSE).

Formula:

$$RMSE = \sqrt{MSE}$$

Where:

• RMSE = Root Mean Squared Error

• MSE = Mean Squared Error

Example:

From the previous example,

MSE = 4

Step 1: Calculate RMSE

$$RMSE = \sqrt{4}$$

$$= 2$$

Final Answer:

RMSE = 2

Interpretation:

• RMSE = 0 → Perfect prediction

• Lower RMSE → Better model

• Higher RMSE → More prediction error

Advantages:

• Uses the same unit as the target (actual) variable.

• Easy to interpret.

• Penalizes larger errors more than MAE because the errors are squared before taking the square root.

10. R^2 Score (Coefficient of Determination)

R^2 Score measures how well the Linear Regression model fits the data.

Formula:

$$R^2 = 1 - [\Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2]$$

Where:

• R^2 = Coefficient of Determination

• y_i = Actual value

• $\hat{y}_i$ = Predicted value

• $\bar{y}$ = Mean of the actual values

Example:

Actual Values (y):

[10, 20, 30]

Predicted Values ($\hat{y}$):

[12, 18, 28]

Step 1: Calculate Mean of Actual Values

$\bar{y}$ = (10 + 20 + 30) / 3

= 60 / 3

= 20

Step 2: Calculate Sum of Squared Errors (SSE)

$\Sigma(y_i - \hat{y}_i)^2$

$= (10 - 12)^2 + (20 - 18)^2 + (30 - 28)^2$

$= 4 + 4 + 4$

$= 12$

Step 3: Calculate Total Sum of Squares (SST)

$\Sigma(y_i - \bar{y})^2$

$= (10 - 20)^2 + (20 - 20)^2 + (30 - 20)^2$

$= 100 + 0 + 100$

$= 200$

Step 4: Calculate R^2 Score

$R^2 = 1 - (12 / 200)$

$= 1 - 0.06$

$= 0.94$

Final Answer:

$R^2 = 0.94$

Interpretation:
- $R^2 = 1$ → Perfect fit
- $R^2 \approx 0$ → Model performs no better than predicting the mean
- $R^2 < 0$ → Model performs worse than predicting the mean

Range:
- 1 → Perfect model
- 0 → Average model
- Less than 0 → Poor model

11. Multiple Linear Regression

```
    Used with multiple features.
    Formula:
    y = b₀ + b₁x₁ + b₂x₂ + … + bₙxₙ
    Example:
    Price = 5000 + 2000(Age) + 300(Size)

    lets
        Age = 10 years
        Size = 1000 sq ft
    so
        Price = 5000 + 2000(10) + 300(1000)
        Price = 5000 + 20000 + 300000
        Price = 325000

12. Vector Form
    Y = Xβ + ε
    Where:
    •   Y = Target
    •   X = Feature Matrix
    •   β = Coefficients
    •   ε = Error
    Example:
    Y = [[200], [300], [400]]
    X = [[1000, 1], [1500, 1], [2000, 1]]
    β = [[0.2], [0]]
    ε = [[0], [0], [0]]

    Y = [200, 300, 400]
    X = [[1000, 1], [1500, 1], [2000, 1]]
    β = [0.20]
    ε = [0, 0, 0]

    Multiple Linear Regression of matrix formula:
    Y = Xβ + ε
    ε = 0
    Y = Xβ
    Y = [[1000, 1],
        [1500, 1],
        [2000, 1]] * [[0.2], [0]]

    1st row:
```

```
    1000 * 0.2 + 1 * 0 = 200
    2nd row:
    1500 * 0.2 + 1 * 0 = 300
    3rd row:
    2000 * 0.2 + 1 * 0 = 400

    So:
    Y = [200, 300, 400]

    so your actual Y matches perfectly.

    That means:

    Coefficient = 0.2
    Intercept = 0
    Error (ε) = 0
    Model is perfectly fit (R² = 1)

    relation is:

    y=0.2x

    That means for every 1 unit increase in x, y increases by 0.2 units.

13. Normal Equation
    Directly calculates coefficients.
    Formula:
    β = (XᵀX)⁻¹Xᵀy
    Meaning:
    •   X = Input Matrix
    •   Xᵀ = Transpose
    •   (XᵀX)⁻¹ = Matrix Inverse
    •   y = Output
    •   β = Weights
    Example:
    X = [[1,1], [1,2], [1,3]]
    y = [[2], [4], [6]]
    Result:
    β = [[0], [2]]
    Equation:
    y = 2x
```

```
14. Gradient Descent
    Iterative optimization.
    Update:
    m = m - α(∂J/∂m)
    c = c - α(∂J/∂c)
    Where: α = Learning Rate
    Large α → Overshoot
    Small α → Slow

15. Assumptions of Linear Regression
    •   Linearity
    •   Independence
    •   Homoscedasticity
    •   Normality
    •   No Multicollinearity

16. Train Test Split
    from sklearn.model_selection import train_test_split
    X_train, X_test, y_train, y_test = train_test_split( X, y,
test_size=0.2, random_state=42 )
    80% Train
    20% Test

17. Build Linear Regression
    from sklearn.linear_model import LinearRegression
    model = LinearRegression()
    model.fit( X_train, y_train )

18. Prediction
    y_pred = model.predict(X_test)

19. Evaluation
    from sklearn.metrics import (mean_absolute_error,
mean_squared_error, r2_score )

20. Coefficients
    print("Coefficients:", model.coef_)
    print("Intercept:", model.intercept_)

21. Visualization
```

~~~
    import seaborn as sns
    import matplotlib.pyplot as plt
    sns.scatterplot( x=y_test, y=y_pred )
    plt.show()

22. Advantages
    ✓ Easy
    ✓ Fast
    ✓ Interpretable
    ✓ Works well for linear data

23. Disadvantages
    ✗ Sensitive to outliers
    ✗ Poor for nonlinear data
    ✗ Overfitting possible

24. Real Project Flow
    Load Dataset ↓
    Cleaning ↓
    Split ↓
    Train ↓
    Predict ↓
    Evaluate ↓
    Deploy
    Short Summary
    Linear Equation:
    y = mx + c
    Multiple:
    y = b₀ + b₁x₁ + …
    Normal Equation:
    β = (XᵀX)⁻¹Xᵀy


    Metrics:
    MAE
    MSE
    RMSE
    R²

'''
~~~